

1. Shor's algorithm, developed by mathematician Peter Shor in 1994, poses a significant threat to widely used cryptography systems, particularly RSA and Elliptic Curve Cryptography (ECC). Both RSA and ECC rely on the computational difficulty of specific mathematical problems. RSA is based on the difficulty of factoring large integers while ECC depends on the difficulty of solving the elliptic curve discrete logarithm problem. Classical computers are unable to solve these problems efficiently, which is why these systems are considered secure today. However, Shor's algorithm, when run on a sufficiently powerful computer, can solve both of these problems in polynomial time. In summary, it undermines the core assumptions that make RSA & ECC secure. Its potential realization threatens the integrity, confidentiality and trust of the entire digital ecosystem, necessitating a swift transition to quantum-safe cryptography.

Q2] Quantum Key Distribution (QKD) is a revolutionary approach to secure communication that uses principles of quantum mechanics to distribute encryption keys between parties in a provably secure manner. Unlike classical public-key encryption, which relies on computational hardness assumptions, QKD derives its security from the laws of physics, specifically the behavior of quantum particles such as photons. The primary role of QKD in future cryptographic systems is to enable information-theoretically secure key exchange, making it immune to attacks from both classical and quantum computers. As quantum computing threatens traditional cryptography, QKD offers a future-proof method of establishing secure communication channels. The key differences are:

- i) Security Basis: It is based on quantum physics.
- ii) Resistance to Quantum attacks: QKD is inherently secure against quantum attacks.
- iii) Key Distribution: QKD requires special quantum channels.
- iv) Scalability: classical are easier to scale globally while QKD faces challenges in long-distance transmission.

BI Lattice-based cryptography and traditional number-theoretic cryptography represent two fundamentally different approaches to securing data. These differences become especially significant in the face of emerging quantum computing technologies.

i) Underlying Hard Problems: RSA/ECC based on number-theoretic problems such as integer factorization and discrete logarithm problem

ii) Quantum Resistance: RSA/ECC vulnerable to Shor's algorithm which can efficiently factor large numbers. Lattice-Based cryptography currently considered quantum resistant as no efficient quantum algorithm needed

iii) Security Guarantees: RSA/ECC it is based on unproven mathematical assumptions. Lattice-based offers strong worst-case to average case hardness.

iv) Performance & Efficiency: RSA/ECC mature and well optimized. Lattice-based requires large key

v) Applications: RSA/ECC used in SSL/TLS digital signatures, encrypted email. Lattice-based supports advanced cryptographic primitives like fully homomorphic encryption.

01/05/21

IT-21010

Q9/ Custom PRNG using Time & Seed (Python code below)

```

import time

class Custom PRNG:
    def __init__(self, need=None):
        if need is None:
            need = int(time.time_ns())
        self.need = need
        self.modulus = 2**32
        self.a = 1669525
        self.c = 10139309223
        self.state = seed
        print(f"[INFO] PRNG initialized with need: {self.need}")

    def next(self):
        self.state = (self.a * self.state + self.c) % self.modulus
        return self.state

    def random(self):
        return self.next() / self.modulus

if __name__ == "__main__":
    custom_seed = int(time.time_ns()) * 123456789
    prng = Custom PRNG(Seed=custom_seed)
    print("Generating 5 pseudo-random numbers...")
    for i in range(5):
        print(f"Random # {i+1}: {prng.random()}")

```

Q5] Prime less than 50 python implementation

```
def sieve_of_eratosthenes(n):
```

```
    in_prime = [True] * n
```

```
    in_prime[0:2] = [False, False]
```

```
    for p in range(2, int(n**0.5)+1):
```

```
        if in_prime[p]:
```

```
            for i in range(p*p, n, p):
```

```
                in_prime[i] = False
```

```
    primes = [i for i, prime in enumerate(in_prime) if prime]
```

```
    return primes
```

```
primes_below_50 = sieve_of_eratosthenes(50)
```

```
print("Prime numbers less than 50:", primes_below_50)
```

Output: prime numbers less than 50:

```
[2, 3, 5, 7, 11, 13, 17, 19, 23, 29, 31, 37, 41, 43, 47]
```

Time Complexity Comparison:

Sieve of Eratosthenes $O(n \log \log n)$

Efficient for generating all primes up to n

Trial Division $O(n^2)$ or $O(n \sqrt{n})$

checks each number for divisibility up to $\sqrt{n}$

Q6/ A Carmichael number is a special type of composite number that fails Fermat's primality test. That is, a composite number n is called a Carmichael number if $a^{n-1} \equiv 1 \pmod n$ for all integers a such that $\gcd(a, n) = 1$.

Necessary & Sufficient Conditions:

n is a Carmichael number if & only if:

- i) n is square free and ii) For every prime divisor p of n , it holds that $p-1 \mid n-1$.

Now For $n = 561$

Prime factorization:

$$561 = 3 \times 11 \times 17$$

all primes are distinct

check condition for each $p \mid 561$

$$p = 3: 3-1 = 2, \text{ and } 2 \mid 560$$

$$p = 11: 11-1 = 10, \text{ and } 10 \mid 560$$

$$p = 17: 17-1 = 16, \text{ and } 16 \mid 560$$

It is a Carmichael number

For $n = 1105$

Prime factorization $1105 = 5 \times 13 \times 17$

check conditions:

$$5-1 = 4, \text{ and } 4 \mid 1104$$

$$13-1 = 12, \text{ and } 12 \mid 1104$$

$$17-1 = 16, \text{ and } 16 \mid 1104$$

It is a Carmichael number

For $n = 1729$

Prime factorization

$$1729 = 7 \times 13 \times 19$$

$$7-1 = 6, \text{ and } 6 \mid 1728$$

$$13-1 = 12, \text{ and } 12 \mid 1728$$

$$19-1 = 18, \text{ and } 18 \mid 1728$$

So, it is a Carmichael number

971 $\mathbb{Z}_{11}$ is a ring because $\mathbb{Z}_{11}$ is the set of integers modulo 11 $\mathbb{Z}_{11} = \{0, 1, 2, \dots, 10\}$ addition & multiplication are defined modulo 11. A ring requires:

$(\mathbb{Z}_{11}, +)$ is an Abelian group: closure, associativity, identity (0), inverse and commutativity are all satisfied.

$(\mathbb{Z}_{11}, \cdot)$ is closed and associative. Distributive property holds $a \cdot (b+c) \equiv a \cdot b + a \cdot c \pmod{11}$.

$(\mathbb{Z}_{37}, +)$ is an Abelian group. Because it consists of integers modulo 37; $\mathbb{Z}_{37} = \{0, 1, \dots, 36\}$

Operation: Addition modulo 37

Group axioms: closure, Associativity, identity, Inverse $\forall a \in \mathbb{Z}_{37}, \exists -a \pmod{37}$, commutativity.

$(\mathbb{Z}_{35}^*, \cdot)$ It is also Abelian group. It is the set of units modulo 35 under multiplication

$$\mathbb{Z}_{35}^* = \{a \in \mathbb{Z}_{35} \mid \gcd(a, 35) = 1\}$$

$35 = 5 \times 7 \Rightarrow$ Not prime so not all elements are invertible

The units are $\mathbb{Z}_{35}^* = \{1, 2, 3, \dots, 34\}$

It is closure, associativity, identity, inverse and commutative

Q8] To find the remainder we compute $-52 \bmod 31$
 Modulo always returns a non-negative remainder between
 0 and $n-1$, we want to find a number r such that
 $-52 \equiv r \bmod 31$ where $0 \leq r < 31$

$$-52 + 31 = -21 \text{ (still negative)}$$

$$-21 + 31 = 10 \text{ (between 0 \& 31)}$$

$$-52 \equiv 10 \bmod 31$$

So, our remainder is 10

Q9] To find the multiplicative inverse of 7 mod 26, we
 want to find an integer x such that $7x \equiv 1 \bmod 26$

$$\text{This is equivalent to finding } 7x + 26y = 1$$

$$\text{Start from } 1 = 5 - 2 \cdot 2$$

$$\text{but from above } 2 = 7 - 1 \cdot 5 \text{ so}$$

$$1 = 5 - 2(7 - 1 \cdot 5) = 5 - 2 \cdot 7 + 2 \cdot 5 = 3 \cdot 5 - 2 \cdot 7$$

$$\text{new replace } 5 = 26 - 3 \cdot 7$$

$$1 = 3(26 - 3 \cdot 7) - 2 \cdot 7 = 3 \cdot 26 - 9 \cdot 7 - 2 \cdot 7 = 3 \cdot 26 - 11 \cdot 7$$

so

$$1 = 3 \cdot 26 - 11 \cdot 7$$

$$\text{final answer } -11 \cdot 7 + 3 \cdot 26 = 1 \Rightarrow x = -11$$

$$-11 \equiv 15 \bmod 26$$

So the multiplicative inverse of 7 mod 26 is 15

Q10) First Multiply $-8 \times 5 = -40$

Reduce modulo 7 we want $-40 \pmod{7}$
 To get a positive remainder, we can add 7
 repeatedly until the result is in the range

$$0 \leq r < 7 \quad -40 + 7 = -33$$

$$-33 + 7 = -26$$

$$-26 + 7 = -19$$

So our result is 11

The General rule is $(a.b) \pmod{m}$

When dealing with negative numbers modulo m
 convert them first using

$$a \pmod{m} = (a \pmod{m}) \pmod{m} \text{ (if } a < 0 \text{)}$$

Q11) Statement of Bezout's Theorem: Let a & b be integers
 not both zero. Then there exist integers x and y such that
 $ax + by = \gcd(a, b)$

If $\gcd(a, m) = 1$, then Bezout's identity tells us there
 exists an x such that

$$ax + my = 1 \Rightarrow ax \equiv 1 \pmod{m}$$

(2) If $\gcd(a, m) = d$, then $ax \equiv 1 \pmod{m}$ has no solution.

We need to solve $07x \equiv 1 \pmod{385}$

which means: $07x + 385y = 1$

we compute $\gcd(385, 07)$:

$$385 = 3 \cdot 07 + 04$$

$$07 = 1 \cdot 04 + 3$$

$$04 = 3 \cdot 3 + 1$$

$$3 = 3 \cdot 1 + 0$$

$\therefore \gcd(07, 385) = 1 = \text{inverse}$

Now Extended Euclidean Algorithm

Starts from: $1 = 07 - 3 \cdot 04$

substitute $3 = 07 - 1 \cdot 04$

$$1 = 07 - 3(07 - 1 \cdot 04) = 07 - 3 \cdot 07 + 3 \cdot 04$$

$$= 3 \cdot 04 - 2 \cdot 07$$

Now substitute $04 = 385 - 3 \cdot 07$

$$1 = 3 \cdot (385 - 3 \cdot 07) - 2 \cdot 07$$

$$= 3 \cdot 385 - 9 \cdot 07 - 2 \cdot 07$$

$$= 3 \cdot 385 - 11 \cdot 07$$

$$1 = 3 \cdot 385 - 11 \cdot 07$$

So our final answer $27(-11) + 385 \cdot 3 = 1 \Rightarrow -11$ is

the inverse of $07 \pmod{385}$

We want the positive inverse, so reduce

$$-11 \pmod{385} = 385 - 11 = 374$$

12] Bezout's identity (existence): Let a, b be integers, not both zero and let $d = \gcd(a, b)$. Run the Euclidean algorithm to compute

$$a = q_1 b + r_1, 0 \leq r_1 < b.$$

$$b = q_2 r_1 + r_2$$

$$r_1 = q_3 r_2 + r_3$$

$$\vdots$$

$$r_{k-2} = q_k r_{k-1} + r_k$$

$$r_{k-1} = q_{k+1} r_k + 0$$

So $r_k = d$. Each remainder r_i is an integer linear combination of a and b because $r_1 = a - q_1 b$ and thereafter $r_{i+1} = r_{i-1} - q_{i+1} r_i$. By back substitution we express r_k as $r_k = x a + y b$ for some integers x, y . Hence d is an integer linear combination of a and b . Since any common divisor of a and b divides every linear combination, d is the smallest positive such combination and we have Bezout's identity: $ax + by = \gcd(a, b)$.

Find x with $93x \equiv 1 \pmod{29}$ we need an integer x with $93x + 29y = 1$. Use the Euclidean algorithm

$$29 = 5 \cdot 93 + 25$$

$$93 = 1 \cdot 25 + 18$$

$$25 = 1 \cdot 18 + 7$$

$$18 = 2 \cdot 7 + 4$$

$$7 = 1 \cdot 4 + 3$$

$$9 = 1 \cdot 3 + 1$$

$$3 = 1 \cdot 3 + 0$$

IT-2010

Back-substitute to express 1 as a combination of 93 & 290

$$1 = 1 - 1 \cdot 0 = 9 - 1 \cdot 3$$

$$= 9 - 1(7 - 1 \cdot 9) = 2 \cdot 9 - 1 \cdot 7$$

$$= 2(18 - 2 \cdot 7) - 1 \cdot 7 = 2 \cdot 18 - 5 \cdot 7$$

$$= 2 \cdot 18 - 5(25 - 1 \cdot 18) = 7 \cdot 18 - 5 \cdot 25$$

$$= 7(93 - 1 \cdot 25) - 5 \cdot 25 = 7 \cdot 93 - 12 \cdot 25$$

$$= 7 \cdot 93 - 12(290 - 5 \cdot 93) = (7 + 60) \cdot 93 - 12 \cdot 290$$

$$= 67 \cdot 93 - 12 \cdot 290$$

So $67 \cdot 93 + (-12) \cdot 290 = 1$. Thus $x = 67$ is a solution and x gives the multiplicative inverse of 93 modulo 290.

13 Fermat's Little Theorem is If p is prime and a is an integer with $\gcd(a, p) = 1$, then $a^{p-1} \equiv 1 \pmod{p}$.

Consider the nonzero residue classes modulo p : $1, 2, \dots, p-1$.

Multiplying each element by a (with $\gcd(a, p) = 1$) permutes this set modulo p . So $a \cdot 1, a \cdot 2, \dots, a \cdot (p-1)$ is congruent

modulo p to some permutation of $1, 2, \dots, p-1$. Taking the product of the terms in both lists gives

$$a^{p-1} \cdot (1 \cdot 2 \cdot \dots \cdot (p-1)) \equiv 1 \cdot 2 \cdot \dots \cdot (p-1) \pmod{p}.$$

Since $1 \cdot 2 \cdot \dots \cdot (p-1)$ is not divisible by p , we may cancel it in the field $\mathbb{Z}/p\mathbb{Z}$, yielding $a^{p-1} \equiv 1 \pmod{p}$.

In 561 prime based on this fact? It is a substitution mod
 Factor $561 = 3 \cdot 11 \cdot 17$. For any a with $\gcd(a, 561) = 1$, by
 Fermat's theorem we have $a^2 \equiv 1 \pmod{3}$, $a^{10} \equiv 1 \pmod{11}$,
 $a^{16} \equiv 1 \pmod{17}$. $F(2, 11, 17) = F(1, (F(5, 11, 17))) =$

Since $\text{lcm}(2, 10, 16) = 80$ & 560 is a multiple of each of
 2, 10, 16 (indeed $560 = 7 \cdot 80$) we get $a^{560} \equiv 1 \pmod{561}$
 each of 3, 11, 17. By the Chinese Remainder Theorem

$$a^{560} \equiv 1 \pmod{561}, \text{ now,}$$

Evaluate $5^{123} \pmod{175}$. Note $175 = 25 \cdot 7$ because $\gcd(5, 175) = 5 \neq 1$. Fermat's theorem does not apply directly to modulus 175.

1. Modulo 25. For $n \geq 2$, $5^n \equiv 0 \pmod{25}$. Hence for any $n \geq 2$

2. Modulo 7. $5^{123} \equiv 0 \pmod{25}$

Here $\gcd(5, 7) = 1$, so we can use Fermat's $5^6 \equiv 1 \pmod{7}$

$$123 \equiv 3 \pmod{6} \text{ (since } 123 = 6 \cdot 20 + 3) \quad 5^{123} \equiv 5^3 \pmod{7}$$

3. Combine CRT to find x . $x \equiv 0 \pmod{25}$, $x \equiv 6 \pmod{7}$

$$k \equiv 2, 6 \equiv 12 \equiv 5 \pmod{7}$$

So $k = 5 + 7t$ and $x = 25k = 25(5 + 7t) = 125 + 175t$

The least residue mod 175 is $x \equiv 125 \pmod{175}$

So the answer is $5^{123} \equiv 125 \pmod{175}$

Q19/Statement: Let $n_1, n_2, \dots, n_k$ be pairwise coprime positive integers. For any integers $a_1, a_2, \dots, a_k$ the system $x \equiv a_i \pmod{n_i}$ has a solution x . Moreover all solutions are congruent modulo $N = n_1 n_2 \dots n_k$. The solution is unique modulo N . Now we have

$$\begin{cases} x \equiv 2 \pmod{3} \\ x \equiv 3 \pmod{5} \\ x \equiv 2 \pmod{7} \end{cases} \quad \text{Here } n_1=3, n_2=5, n_3=7. \text{ They are pairwise coprime. Compute } N=3 \cdot 5 \cdot 7=105$$

Compute N_i :

$$N_1 = 105/3 = 35, \quad N_2 = 105/5 = 21, \quad N_3 = 105/7 = 15.$$

Find inverses j_i with $N_i j_i \equiv 1 \pmod{n_i}$:

For $n_1=3$: $35 \equiv 2 \pmod{3}$. Inverse of 2 mod 3 is 2 (Since $2 \cdot 2 = 4 \equiv 1 \pmod{3}$).

For $n_2=5$: $21 \equiv 1 \pmod{5}$. Inverse of 1 is 1. So $j_2 = 1$.

For $n_3=7$: $15 \equiv 1 \pmod{7}$. Inverse is 1. So $j_3 = 1$.

Form the solution: $x = \sum_{i=1}^3 a_i N_i j_i = 2 \cdot 35 \cdot 2 + 3 \cdot 21 \cdot 1 + 2 \cdot 15 \cdot 1$

calculate: $x = 2 \cdot 70 + 63 + 30 = 140 + 63 + 30 = 233$

Reduce modulo $N = 105$

$$233 \equiv 233 - 2 \cdot 105 = 233 - 210 = 23 \pmod{105}.$$

So we get all of this congruences hold

$$23 \pmod{3} = 2$$

$$23 \pmod{5} = 3$$

$$23 \pmod{7} = 2$$

So our final answer is $x \equiv 23 \pmod{105}$

15] The CIA triad is a functional model in information security that represents the three key principles needed to build and maintain secure system.

i) Confidentiality: Ensures the sensitive information is accessible only to authorized users and not disclosed to unauthorized individuals. Protects privacy and prevents data breaches. Encryption, access controls and data classification are technologies.

ii) Integrity: Guarantees that info remains accurate, complete and unaltered except by authorized changes. Prevents unauthorized modification or corruption of data. Checksums, hashing, digital signatures, version control. Verifying file integrity using a hash value after downloading software.

iii) Availability: Ensures that info and systems are accessible to authorized users when needed. Prevents disruptions or service that could cause business or operational failures. Techniques are Redundancy, load balancing, regular backups, disaster recovery. A website remaining online during high traffic using scalable cloud infrastructure.

16] Difference between Steganography & Cryptography

Cryptography

i) Scrambles the message so that even if intercepted, it can't be understood without a key

ii) The msg is visible but unreadable without decryption

iii) Mathematical algorithm

iv) Example: HELLO $\rightarrow$ JHA901

Steganography

i) Hides the existence of the message so that no one knows it's there

ii) Msg is invisible, hidden inside other harmless-looking data

iii) Concealment methods

iv) HELLO hidden inside a photo

Common Steganography Techniques for Digital Media

A] Image Steganography: LSB Insertion, Palette-based Hiding, Transform Domain Techniques

B] Audio Steganography: LSB coding, Phase coding, Echo Hiding

C] Video Steganography: Frame-by-Frame LSB, Motion Vector Alteration

D] Text Steganography: Whitespace Manipulation, Font or Character substitution

17] II Phishing :

Method $\Rightarrow$ A social engineering attack that tricks users into revealing sensitive data. Usually carried out via fraudulent emails, fake websites, text msg or calls that appear legitimate.

Impact $\Rightarrow$ compromise confidentiality by stealing personal or organizational data. Can lead to identity theft, financial loss, and unauthorized account access.

2] Malware :

Method $\Rightarrow$ Malicious software designed to infiltrate and damage systems. Spread via infected files, malicious downloads, email attachments.

Impact $\Rightarrow$ can compromise confidentiality, integrity

3] Denial-of-Service (DoS) :

Method $\Rightarrow$ Flood a server, network, or application with excessive requests making it unavailable to legitimate users.

Impact $\Rightarrow$ mainly affects availability of service causes downtime, loss of business & reputational damage.

18] The General Data Protection Regulation (GDPR) is a legal framework in the European Union designed to strengthen data privacy, data security and user rights. Although GDPR is not a direct cybersecurity tool, it helps mitigate cyber attacks and protect user privacy in several important ways.

1] Data Protection by Design & by Default: Organizations must integrate privacy & security measures into systems from the start. Reducing the risk of data breaches because sensitive information is better secured before attackers can reach it.

2] Strict Data Minimization & Purpose Limitation: Only collect and store the minimum data necessary for specific purposes. Even if a breach occurs, the amount of data exposed is reduced, limiting attacker gains.

3] Security Measures & Breach Notification: Implement technical and organizational measures. Faster responses help contain the attack & prevent further damage.

4] Accountability & Heavy Penalties: Organizations must keep proof of compliance. The risk of huge fines motivates companies to take security seriously.

5] User Rights & Transparency: Users have rights to access, correct, delete and transfer their data. Makes it hard for attackers or malicious insiders to exploit hidden or unnecessary data stores.

IT-21010

Step 1: Initial permutation (IP): The 64-bit plaintext is rearranged according to a fixed table. Two 32-bit halves; L_0 = left 32 bits, R_0 = right 32 bits

Step 2: Key Preparation: Input 64-bit K_0 → remove 8 parity bits. 56-bit key. Perform permuted choice 1 (PC-1) to rearrange bits. For each round: Left shift C & D (Shifts vary per round by 2 bits). Combine C & D → apply permuted choice 2 (PC-2) → 48-bit subkey.

Step 3: 16 Feistel Rounds: Each round takes L_{i-1} , R_{i-1} & K_i to produce L_i , R_i : Permutation (P) Rearrange the 32 bits for more diffusion: $L_i = R_{i-1}$, $R_i = L_{i-1} \oplus F(R_{i-1}, K_i)$

Step 4: Final Permutation (FP): After 16 rounds, combine R_{16} & L_{16} . Apply Final permutation to produce ciphertext.

Example for msg plaintext: $P = \text{0x0123456789ABCDEF}$
Key $K = \text{0x133457799BBDDFF1}$

Process:

1) IP: Rearrange plaintext bits

2) Round 1: Expand R_0 (32 bits) → 48 bits. XOR with K_1 . Pass through S-boxes → 32 bits. P-box permutation. XOR with L_0

3) Repeat for 16 rounds, each time with a different key

4) FP: Final permutation gives ciphertext

Final output: Ciphertext = 0x8BE81359DF0B915

20) We are given,

$$R_0 = 0 \times \text{F0F0F0F0}$$

$$K_1 = 0 \times \text{0F0F0F0F}$$

$$L_0 = 0 \times \text{AAAAAAAA}$$

$f(R_0, K_1)$ is defined here as the XOR $R_0 \oplus K_1$.

For one DES round: $L_1 = R_0$, $R_1 = L_0 \oplus f(R_0, K_1)$

Compute f :

$$\begin{aligned} f(R_0, K_1) &= R_0 \oplus K_1 = 0 \times \text{F0F0F0F0} \oplus 0 \times \text{0F0F0F0F} \\ &= 0 \times \text{FFFFFFFFFF} \end{aligned}$$

Each byte: $(0 \times \text{F0} \oplus 0 \times \text{0F} = 0 \times \text{FF})$.

Then $L_1 = R_0 = 0 \times \text{F0F0F0F0}$

$$\begin{aligned} R_1 &= L_0 \oplus f(R_0, K_1) = 0 \times \text{AAAAAAAA} \oplus 0 \times \text{FFFFFFFF} \\ &= 0 \times \text{55555555} \end{aligned}$$

21) Computing AES subbytes for the input word

$$[0 \times 23, 0 \times \text{A7}, 0 \times 4\text{C}, 0 \times 1\text{D}]$$

using the standard AES sbox

IT-21010

sbox = [

0x63, 0x7c, 0x77, 0xf2, 0x6f, 0x2f, 0xc8, 0x67, 0xf9, 0xc4, 0xa7, 0x2b, 0x76, 0x6a, 0x35, 0x01, 0x73, 0x6d, 0xca, 0x62, 0xe8, 0x3f, 0xb6, 0x09, 0x04, 0x92, 0x76, 0x29, 0xbd, 0x63, 0xb7, 0x96, 0xcd, 0xb2, 0x09, 0x64, 0x76, 0x59, 0xdf, 0x61, 0xc9, 0x9b, 0xef, 0xc6, 0x53, 0xa6, 0x9d, 0x7d, 0x29, 0x52, 0xd7, 0x2e, 0xf9, 0xc7, 0xd0, 0xf7, 0x94, 0x76, 0x67, 0x35, 0xe4, 0xf9, 0x92, 0xd9, 0x51, 0x2e, 0xd7, 0x6d, 0x6a, 0x97, 0x56, 0x2f, 0x35, 0x29, 0xcd, 0xf6, 0x82, 0x95, 0x6a, 0x97, 0x56, 0x2f, 0x35, 0x29, 0xe7, 0xc7, 0x28, 0x3d, 0x2c, 0x2f, 0x7d, 0x2f, 0x6c, 0x0e]

Input_word = [0x23, 0xA7, 0x9C, 0x19]

output = [sbox[b] for b in input_word]

print("Input word: ", [hex(x) for x in input_word])

print("SubBytes Output: ", [hex(x) for x in output])

Using the standard AES S-box, substitute each byte

0x23 → 0x26

0xA7 → 0x5C

0x9C → 0x29

0x19 → 0xD4

So the resulting output word after subbyte is

[0x26, 0x5C, 0x29, 0xD4]

2) The AddRoundKey step in AES is just a byte-wise XOR between the input word & the round key word. we calculate byte by byte.

1) $0xA \oplus 0x55$

$0xA = (00011010)_2$ $0x55 = (01010101)_2$

XOR: $(01001111)_2 = 0x4F$

2) $0x2B \oplus 0x66$

$0x2B = (00101011)_2$ $0x66 = (01100110)_2$

XOR: $(01001101)_2 = 0x4D$

3) $0x3C \oplus 0x77$

$0x3C = (00111100)_2$ $0x77 = (01110111)_2$

XOR: $(01001011)_2 = 0x4B$

4) $0x4D \oplus 0x88$

$0x4D = (01001101)_2$ $0x88 = (10001000)_2$

XOR: $(11000101)_2 = 0xC5$

So the output word after AddRoundKey:

$[0x4F, 0x4D, 0x4B, 0xC5]$

IT-21010

28) Given Matrix (used by Mix Columns over $GF(2^8)$)

$$M = \begin{bmatrix} 02 & 03 & 01 & 01 \\ 01 & 02 & 03 & 01 \\ 01 & 01 & 02 & 03 \\ 03 & 01 & 01 & 02 \end{bmatrix}$$

Input column (a bytes):

$$a = \begin{bmatrix} a_0 \\ a_1 \\ a_2 \\ a_3 \end{bmatrix} = \begin{bmatrix} 0x01 \\ 0x02 \\ 0x03 \\ 0x04 \end{bmatrix}$$

Also convert each input byte to 8-bit binary

$$0x01 = 00000001_2$$

$$0x02 = 00000010_2$$

$$0x03 = 00000011_2$$

$$0x04 = 00000100_2$$

Compute each output byte: The product is $r = Ma$. Row-wise

$$r_0 = 02 \cdot a_0 \oplus 03 \cdot a_1 \oplus 01 \cdot a_2 \oplus 01 \cdot a_3$$

$$02 \cdot a_0 = 02 \cdot 0x01 = 0x02 \rightarrow 00000010$$

$$03 \cdot a_1 = (02 \cdot 0x02) \oplus 0x02. \text{ First } 02 \cdot 0x02 = 0x04 (00000100),$$

$$\text{Then } 0x04 \oplus 0x02 = 0x06 (00000110)$$

$$a_1 \cdot a_2 = 0x03 \rightarrow 00000011$$

$$a_1 \cdot a_3 = 0x04 \rightarrow 00000100 \quad \text{So, } r_0 = 0x03$$

Computing all r_1, r_2, r_3 we get

$P_0 = 0x03, P_1 = 0x09, P_2 = 0x09, P_3 = 0x0A$

The transformed column after MixColumn is:

$$\begin{bmatrix} 0x03 \\ 0x09 \\ 0x09 \\ 0x0A \end{bmatrix}$$

In Binary $[00000011, 00000100, 00001001, 00001010]$

29] OFB model in a stream cipher mode of AES instead of encrypting the plaintext directly with AES, it repeatedly encrypts an initial value (IV) and uses the result as a keystream to XOR with the plaintext.

How it works: let's define:

Key: AES secret key, IV: Initial vector, $E_K()$: AES encryption function with key K , $\oplus$: XOR operation

Encryption Steps:

1) Set $O_0 = IV$. For block i : $O_i = E_K(O_{i-1})$ (Encrypt the previous output to get the next keystream block)

$C_i = P_i \oplus O_i$ (XOR keystream with plaintext to get cipher)

Decryption Steps:

1) Use the same IV and Key

Generate the same key stream: $O_i = E_K(O_{i-1})$

Recover plain text $P_i = C_i \oplus O_i$

IT-25010

Diagram: (Exemption)

IV $\rightarrow$ AES-01-XOR with P1-C1

↓

↓
AES-02 - XOR with $R_2 - C_3$

↓

↓
AES-03 - XOR with $P_3 - C_3$

— — — — —

(Description)

$I \vee \rightarrow AES - \text{ol} - \text{xor with } c_1 - p_1$

↓

↓
AES-OR - XOR with $C_2 - P_2$

↓

$\downarrow$
AES - 03 - XOR with $S_3 - P_3$

—

25] Error propagation means that a bit error in a single ciphertext block can cause multiple bits in the decrypted plaintext to be wrong. This happens because some AES modes use feedback or chaining between blocks.

Decryption Process For CBC decryption is

$$P_i = A_k(C_i) \oplus C_{i-1}$$

Where $D_c(C_i)$ = block decrypted with AES

C_{i-1} = previous ciphertext block (or IV for first block)

Impact: one ciphertext bit error -

Block P_i completely wrong Block P_{i+1} one bit wrong

Example: Let's say C_2 gets a 1 bit error

$$P_2 = D_c(C_2) \oplus C_1 \text{ (garbled)}$$

$$P_3 = D_c(C_3) \oplus C_2' \text{ (1 bit error)}$$

CFB Mode (Cipher Feedback)

Decryption Process (CFB with block sizes)

$$P_i = C_i \oplus K(C_{i-1})$$

Error effect:

If a single bit in C_i is wrong: The corresponding bit in P_i will be wrong. The error also propagates for the next few blocks depending on feedback sizes because C_i is used as input to AES encryption for future plaintext recovery.

Impact:

In CFB-128 one ciphertext bit error - current block 1 bit error next block: full block corrupted

In smaller feedback sizes error persists for a smaller number of bytes.

23] Justification:

1] Parallelism: CTR mode allows encryption and decryption to be done in parallel because each block is encrypted with a unique counter value, independent of other blocks. CBC mode cannot be fully parallelized during encryption because each plaintext block must be XORed with the previous ciphertext block encryption. ECB mode is parallelizable but insecure.

2] Security: ECB mode is insecure for large files because identical plaintext blocks produce identical ciphertext blocks, revealing patterns in the data. CBC mode hides patterns but is slower for encryption due to dependency on the previous block. CTR mode hides patterns effectively and uses a lightweight approach, making it secure.

3] Error Propagation: CBC mode has error propagation; a single bit error in one ciphertext block affects two plaintext blocks on decryption. CTR mode has minimal error propagation; an error affects only the corresponding bits in the affected block.

27] Given that,
 Message $M = 1$, Public Key $e = 5$, $n = 17$
 private key $d = 11$

RSA encryption formula $C \equiv M^e \pmod{n}$

$$C \equiv 1^5 \pmod{17}$$

$$C \equiv 1 \pmod{17}$$

$$C \equiv 1$$

$\therefore$ ciphertext $= 1$

RSA decryption formula: $M \equiv C^d \pmod{n}$

$$M \equiv 1^{11} \pmod{17}$$

$$M \equiv 1 \pmod{17}$$

$$M \equiv 1$$

Decrypted Message $= 1$

So our ciphertext $= 1$ and our

decrypted message $= 1$

same as original

28] To generate the RSA digital signature S we compute $S \equiv H(m)^d \pmod{n}$

Given that

$$H(m) = 5, d = 3, n = 33$$

compute 5^3 and reduce mod 33

$$5^2 = 25 \pmod{33}, 5^3 = 25 \cdot 5 = 125$$

$$\text{Reduce } 125 \pmod{33} \quad 33 \times 3 = 99, 125 - 99 = 26$$

$\therefore$ Our S is 26

29] we can compute the public key in Diffie-Hellman using

$$\text{public key} = g^{\text{private key}} \pmod{p}$$

Given: $p = 17, g = 3, \text{private key } a = 4 \text{ (Aleg)}$

private key $b = 5 \text{ (Badel)}$

Aleg's public key (A):

$$A = g^a \pmod{p} = 3^4 \pmod{17}$$

$$3^2 = 9$$

$$3^4 = 9^2 = 81$$

$$81 \pmod{17} = 81 - (17 \times 4) = 81 - 68 = 13$$

$\therefore$ Aleg's public key = 13

01/09/2010

Badel's Public Key (B):

$$B = g^b \text{ mod } p = 3^5 \text{ mod } 17$$

$$3^4 = 81 \text{ mod } 17 = 13 \text{ (from above)}$$

$$3^5 = 13 \times 3 = 39$$

$$39 \text{ mod } 17 = 39 - (17 \times 2) = 39 - 34 = 5$$

Badel's public key is 5

2) Ascii values A=65 & B=66

calculate hash for "AB"

$$H("AB") = (65 + 66) \text{ mod } 100$$

$$= 131 \text{ mod } 100 = 31$$

Calculate hash for "BA"

$$H("BA") = (66 + 65) \text{ mod } 100$$

$$= 131 \text{ mod } 100 = 31$$

Both AB & BA produce same hash value 31

This shows a hash collision two different inputs produce the same output. For this hash function, the order of characters doesn't matter because it only sums ASCII values. This makes it very weak.

Q1) IT-21610

31) Formula in MAC = (Message + Secret Key) mod 17

Given values Message = 15 Secret Key = 7

Compute MAC $MAC = (15 + 7) \text{ mod } 17$
 $= 22 \text{ mod } 17 = 5$

MAC = 5 Attacker changes message to 10

The new MAC should be

$$MAC = (10 + 7) \text{ mod } 17$$
$$= 17 \text{ mod } 17 = 0 \quad (\text{but he doesn't know the key } 7)$$

Can they forge the MAC easily? $\Rightarrow$ Yes this MAC Scheme is weak. Because:

This operation is linear $MAC = (msg + Secret Key) \text{ mod } 17$
If the attacker knows the original message (15) and its MAC (5), they can compute the secret key easily.

$$Secret Key = (MAC - Message) \text{ mod } 17 = (5 - 15) \text{ mod } 17$$

Once they have the key, they can generate a valid MAC

for any modified message.

This shows poor security as single known MAC pair reveals the secret key.

IT-21010

32] The TLS handshake is the process by which a client and a server agree on encryption parameters & securely generate a shared symmetric session key. This key is then used for fast and secure data transfer, while asymmetric cryptography is used only during the handshake to establish that key securely.

Steps in the TLS handshake

Step 1: client Hello: The client sends $\rightarrow$ supported TLS version, supported cipher suites, random number, optional extensions

Step 2: Server Hello: The server responds with $\rightarrow$ chosen TLS version, selected cipher suite, random numbers, digital certificate

Step 3: Server Authentication: The client verifies $\rightarrow$ certificate validity, domain name matches, certificate hasn't expired or been revoked. This ensures the client is talking to the real server, not an impostor.

Step 4: Key Exchange: There are two common methods for that (a) RSA key exchange (older), (b) Diffie-Hellman/ECDHE

Step 5: Session Key Derivation: Both parties now have - client random, server random, pre-master secret. They feed those into a key derivation function (KDF) to produce symmetric encryption keys, message authentication keys.

Step 6: Finished Messages: Both client & server send an encrypted "Finished" message using the newly derived symmetric keys. If the other side decrypts it successfully the handshake is complete.

IT-21010 clauderai

B3] SSH (Secure Shell) uses a three-layer protocol stack to securely connect two systems over an insecure network.

SSH Layered Architecture

1) Transport Layer (SSH-TRANS)

Establishes the secure channel between client & server. handles encryption, manage integrity and server authentication.

Functions: negotiates encryption algorithms. Negotiates manage authentication codes for integrity. Authenticates the server using its host key.

2) User Authentication Layer (SSH-AUTH)

Verifies the identity of the client to the server.

Functions: Supports multiple authentication methods. Password authentication, public key authentication.

Runs after the secure channel is established in the transport layer.

3) Connection Layer (SSH-CONN)

Manages multiple logical channels over the single encrypted SSH connection. Function: Enables port forwarding.

Provides interactive shell sessions.

Supports file transfer. Allows multiplexing multiple streams over one TCP connection.

31) The TLS handshake is the process by which a client & a server - Agree on encryption parameters. Authenticate each other. Securely generate a shared symmetric session key for encrypted communication.

Steps in the TLS Handshake :

Step 1 (ClientHello) → The client sends supported TLS version, List of supported cipher suites, Random number, optional extensions.

Step 2 (ServerHello) → The server replies with chosen TLS version, selected cipher suite, Random number, Server's digital certificate.

Step 3 (Server Authentication) → The client verifies the server's certificate chain, CA signature, matches domain value, ensures it's valid.

Step 4 (Key Exchange) → Two main methods 1) RSA where client generates a pre-Master secret. Encrypts it with the server's public key. Sends it to the server, which decrypts it with private key.

2) (ECDHE preferred) → Client & server exchange ephemeral public keys. Both compute the same shared secret using Diffie-Hellman math. Providing forward secrecy.

IT-2/0/0

Step 5: Session Key Generation $\rightarrow$ Both parties now have client random, server random, pre-master secret. They run these through a key derivation function (KDF) to generate symmetric encryption keys.

Step 6: Finished Messages $\rightarrow$ Client & server send "Finished" message encrypted with the new symmetric keys. Both managers decrypt and verify correctly, the handshake is complete.

Summary Flow

Client Hello $\rightarrow$ Server Hello $\rightarrow$ Certificate $\rightarrow$ Key Exchange $\rightarrow$ Session Key $\rightarrow$ Finished

351 General Form of an Elliptic Curve Equation

Over a finite field $\mathbb{F}_p$ (where p is a prime) the general short Weierstrass form is

$$y^2 \bmod p = (x^3 + ax + b) \bmod p$$

where $a, b \in \mathbb{F}_p$ are constants defining the curve

The curve must satisfy the non-singularity condition

$$4a^3 + 27b^2 \not\equiv 0 \pmod{p}$$

IT-21010

to ensure no loops or self-intersections. The set of points (x, y) that satisfy the equation plus a special point at infinity form an abelian group under a defined addition operation.

Elliptic curves are used in cryptography because

- 1) High security per bit: ECC offers the same security as RSA but with much smaller key sizes.
- 2) Based on the Elliptic Curve Discrete Logarithm Problem: Given two points P & $Q = kP$ on the curve, it's computationally infeasible to find k (Private Key) if the field size is large.
- 3) Efficient Computation: Smaller keys mean faster encryption, decryption and signature operations.
- 4) Lower Bandwidth and storage requirements: Useful for constrained devices.

$$y^2 = x^3 + ax + b \pmod{p}$$

is used because it defines a mathematical structure with a hard to reverse problem, enabling strong, efficient public-key cryptography.

36 | ECC (Elliptic Curve Cryptography) achieves the same security as RSA with a smaller key size because the mathematical problem it's based on is much harder.

RSA vs ECC Security Basis: RSA Security is based on the difficulty of factoring large integers. ECC Security is based on the Elliptic curve Discrete logarithm problem (ECDLP). Given two points P and $Q = kP$ on an elliptic curve, it's computationally infeasible to find k .

Handson Difference: The best known algorithms for factoring (RSA) are much faster than the best known algorithms for solving ECDLP. This means ECC can use much smaller numbers to achieve the same security level.

Key size Comparison

Security Level	RSA Key Size	ECC Key Size
128 bit Sec	3072 bits	256 bits
192 bit Sec	7680 bits	384 bits
256 bit Sec	15360 bits	521 bits

Benefits of smaller ECC keys: Faster computations
Lower bandwidth, Less storage
Better for mobile/IoT devices.

37] we are given:

$$y^2 = x^3 + 2x + 3 \pmod{97}$$

Point $P = (3, 6)$.

Step 1: check LHS

$$y^2 = 6^2 = 36 \pmod{97}$$

Step 2: check RHS

$$x^3 + 2x + 3 = 3^3 + 2(3) + 3 = 27 + 6 + 3 = 36 \pmod{97}$$

$$\text{So RHS} \equiv 36 \pmod{97}$$

Step 3: Compare

$$\text{L.H.S} \equiv \text{R.H.S} \equiv 36 \pmod{97}$$

Yes, the point $(3, 6)$ lies on the curve

38] we are given

$$p = 23$$

$$g = 5$$

$$h = 8 \text{ (Public key } = g^h \pmod{p})$$

message $m = 10$

Random $k = 6$

Compute C_i

$$C_i = g^k \pmod{p}$$

$$C_1 = 5^6 \pmod{23}$$

IT-21010

First calculate 5^6 :

$$5^2 = 25 \equiv 2 \pmod{23}$$

$$5^4 = (5^2)^2 \equiv 2^2 = 4 \pmod{23}$$

$$5^6 = 5^4 \cdot 5^2 \equiv 4 \cdot 2 = 8 \pmod{23}$$

So, $C_1 = 8$

compute $C_2 = m \cdot h^k \pmod{p}$

First find $h^k \pmod{p}$: $h^k = 8^6 \pmod{23}$

$$8^2 = 64 \equiv 18 \pmod{23}$$

$$8^4 = 18^2 = 324 \equiv 2 \pmod{23}$$

$$8^6 = 8^4 \cdot 8^2 \equiv 2 \cdot 18 = 36 \equiv 13 \pmod{23}$$

Now $C_2 = 10 \cdot 13 \pmod{23}$

$$C_2 = 130 \equiv 15 \pmod{23}$$

The ElGamal ciphertext is

$$(C_1, C_2) = (8, 15)$$

So the encrypted message is

$$(8, 15)$$

01.07.2010

39] Importance of Lightweight Cryptography for IoT
IoT devices often have limited processing power, limited memory, low energy availability, wireless communication. Because of these constraints, traditional cryptography like RSA or standard AES in full mode can be too resource-intensive.

Lightweight cryptography is designed to use smaller key sizes, require fewer CPU cycles, minimize memory usage, consume less energy, still protect against eavesdropping, tampering, and forging.

It is Important for IoT Because:

- 1) Security without sacrificing Performance, 2) Scalability
- 3) Longer battery life, 4) Protection in resource-constrained environment

Example Lightweight Encryption Algorithm

- 1) A block cipher designed for low-resource devices
- 2) Block size 64 bits
- 3) Key sizes 80 or 128 bits
- 4) Uses a substitution-permutation Network (SPN)
- 5) Extremely low memory footprint

Other examples: SPECK, SIMON, LEA, KATAN and TinyAES

40] Three Common IoT-Specific Attacks & Mitigation

1) Firmware Hijacking: Attackers modify or replace the IoT device's firmware with malicious code, allowing them to take control or steal data. Unauthorized device control. Backdoor for persistent attacks. Use digitally signed firmware updates. Enable secure boot so only trusted firmware can run. Regularly update firmware from trusted sources.

2) Physical Tampering: Attackers physically access the IoT device to steal keys, modify hardware or attach debugging tools to extract sensitive info. Compromised cryptographic keys. Hardware damage or sabotage. Use tamper-resistant hardware. Store keys in secure elements or TPM chips. Place devices in physically secure locations where possible.

3) IoT Botnets: Malware infects IoT devices with weak/default passwords, adding them to a botnet for large-scale distributed denial of service (DDoS) attacks. Network outages due to massive traffic floods.

IT-21010

Devices used in criminal activities without owner's knowledge: change default credentials, immediately after deployment. Implement strong authentication. Restrict unnecessary internet exposure. Keep firmware patched to ~~known~~ known vulnerabilities.